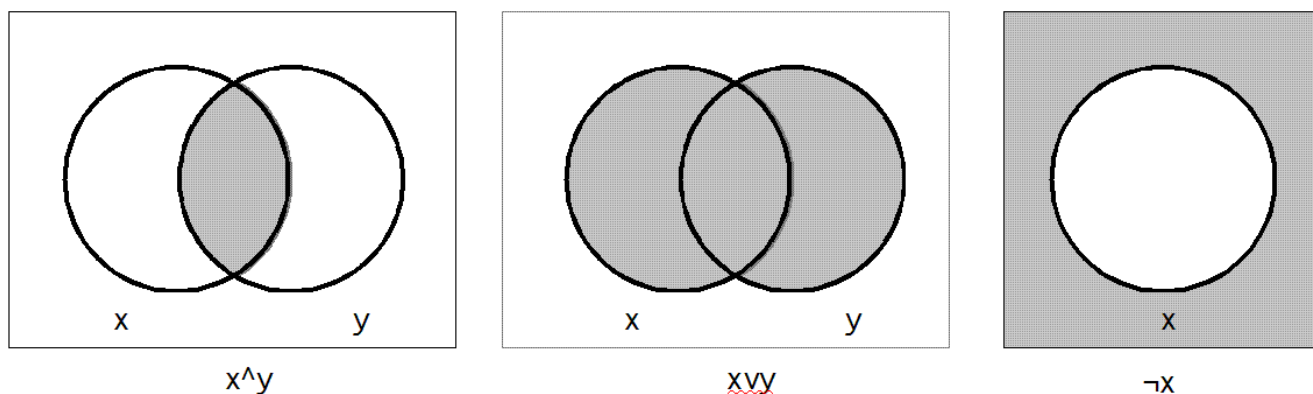




Set in Python

Python's built-in set type has the following characteristics:

- Sets are **unordered**.
- Set elements are **unique**. Duplicate elements are not allowed.
- A set itself may be modified, *but the elements contained in the set must be of an immutable type*. It can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have mutable elements like lists, sets or dictionaries as its elements.

Creating set

In [14]:

```
s1={1,2,3,3}
print(s1)
s1={"hello",3.3,1}
print(s1)
s1={}
print(s1)
print(type(s1))
s1=set()
print(type(s1))
# s1=[1,2,3] gives error because list is not allowed to be member of set
l1=[1,2,3,1]
s1=set(l1)
print(s1)
```

```
{1, 2, 3}
{1, 3.3, 'hello'}
{}
<class 'dict'>
<class 'set'>
{1, 2, 3}
```

In [2]:

```
#order of element does not matter
s1={1,2,3}
s2={2,1,1,3}
print(s1==s2)
print(s2)
```

True

{1, 2, 3}

Modifying a set in Python

- Sets are mutable. However, since they are unordered, indexing has no meaning. We cannot access or change an element of a set using indexing or slicing. Set data type does not support it.
- We can add a single element using the `add()` method, and multiple elements using the `update()` method. The `update()` method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

In [3]:

```
s1={1,2,3}
s1.add(4) # add one element
print(s1)
s1.update([5,6,7])
print(s1)
```

{1, 2, 3, 4}

{1, 2, 3, 4, 5, 6, 7}

Removing elements from a set

- `discard()` function leaves a set unchanged if the element is not present in the set.
- `remove()` function will raise an error in such a condition (if element is not present in the set).

In [15]:

```
s1={1,2,3,4,5,6}
s1.discard(2)
print(s1)
s1.discard(2)
print(s1)
s1.remove(3)
print(s1)
#s1.remove(3) will raise error as 3 is not there in set
#print(s1)
```

{1, 3, 4, 5, 6}

{1, 3, 4, 5, 6}

{1, 4, 5, 6}

pop() and clear()

- Since set is an unordered data type, there is no way of determining which item will be popped. It is completely arbitrary.
- We can also remove all the items from a set using the `clear()` method.

In [5]:

```
s1=set("hello world")  
print(s1)  
print(type(s1))  
s1.pop()
```

```
{'w', 'l', 'd', ' ', 'e', 'r', 'o', 'h'}  
<class 'set'>
```

Out[5]:

```
'w'
```

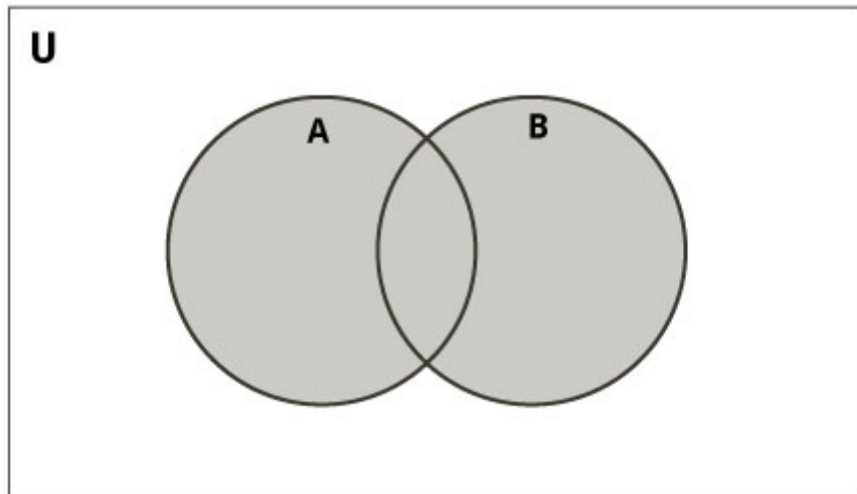
Python Set Operations

- Set Union
- Set Intersection
- Set Difference

In [6]:

```
A = {1, 2, 3, 4, 5}  
B = {4, 5, 6, 7, 8}  
print(A | B)  
print(A.union(B))
```

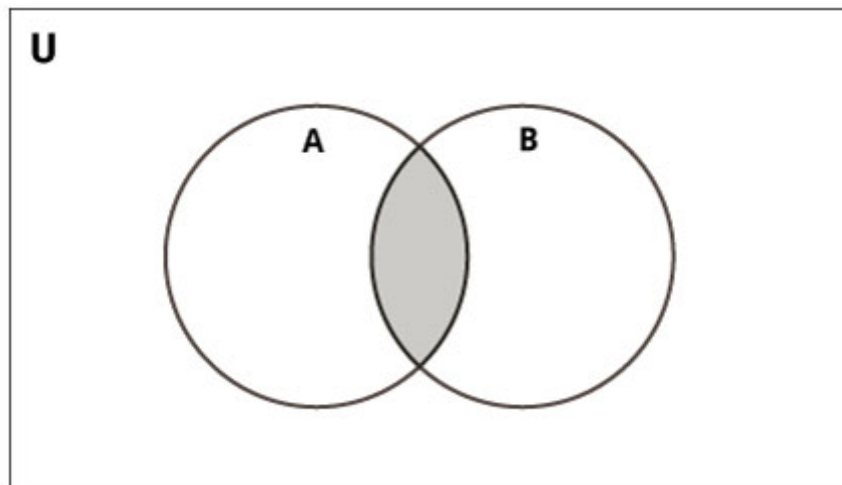
```
{1, 2, 3, 4, 5, 6, 7, 8}  
{1, 2, 3, 4, 5, 6, 7, 8}
```



In [7]:

```
A = {1, 2, 3, 4, 5}  
B = {4, 5, 6, 7, 8}  
print(A & B)  
print(A.intersection(B))  
print(B.intersection(A))
```

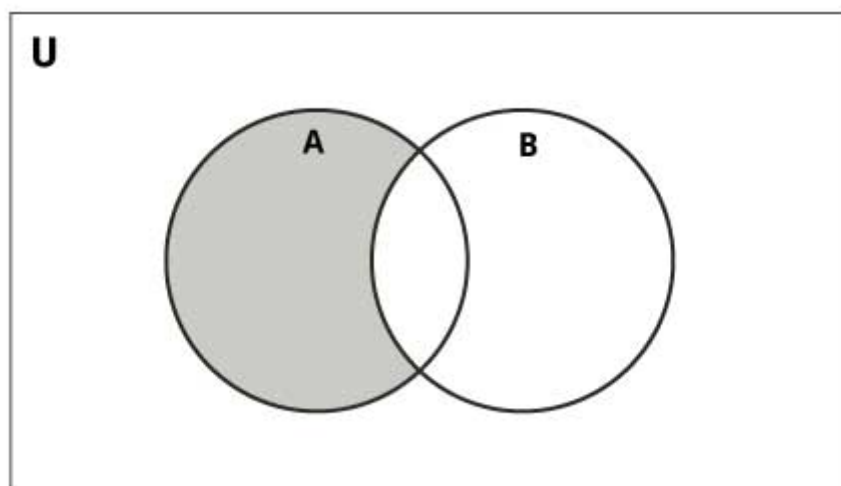
```
{4, 5}  
{4, 5}  
{4, 5}
```



In [8]:

```
#set difference
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
print(A - B)
print(B-A)
print(A.difference(B))
print(B.difference(A))
```

```
{1, 2, 3}
{8, 6, 7}
{1, 2, 3}
{8, 6, 7}
```



In [9]:

```
#Symmetric Difference of A and B is a set of elements in A and B but not in both (excluding  
#Symmetric difference is performed using ^ operator.
```

```
A = {1, 2, 3, 4, 5}  
B = {4, 5, 6, 7, 8}  
print(A ^ B)  
print(A.symmetric_difference(B))  
print(B.symmetric_difference(A))  
x={1,2,3}  
y={1,2,3,4}  
print(x.issubset(y))  
print(y.issuperset(x))
```

```
{1, 2, 3, 6, 7, 8}
```

```
{1, 2, 3, 6, 7, 8}
```

```
{1, 2, 3, 6, 7, 8}
```

```
True
```

```
True
```

In [10]:

```
#accessing elements of set  
s1={"hello world"}  
for i in s1:  
    print(i)
```

```
hello world
```

Built in Set functions

In [11]:

```
A = {1, 2, 6, 4, 5}  
print("len=",len(A))  
print("max=",max(A))  
print("min=",min(A))  
print("sorted=",sorted(A))  
print("sum array=",sum(A))
```

```
len= 5
```

```
max= 6
```

```
min= 1
```

```
sorted= [1, 2, 4, 5, 6]
```

```
sum array= 18
```

Python Frozenset

- Frozenset is a new class that has the characteristics of a set, but its elements cannot be changed once assigned. While tuples are immutable lists, frozensets are immutable sets.
- Sets being mutable are unhashable, so they can't be used as dictionary keys. On the other hand, frozensets are hashable and can be used as keys to a dictionary.
- Frozensets can be created using the frozenset() function.

In [12]:

```
A = frozenset([1, 2, 3, 4])
B = frozenset([3, 4, 5, 6])
print(A.isdisjoint(B))
print(A|B)
print(A&B)
```

False

frozenset({1, 2, 3, 4, 5, 6})

frozenset({3, 4})

In [13]:

```
#A.add(2) raise error that 'frozenset' object has no attribute 'add'
```

Web References:

- <http://www.python.org/> (<http://www.python.org/>)
- <https://www.learnpython.org/> (<https://www.learnpython.org/>)
- <http://www.w3schools.com/> (<http://www.w3schools.com/>)

###

 click me

 Markdown Logo.

In [2]:

```
%pwd
```

Out[2]:

'C:\\Users\\student'

In []: